

Frequency-Corrupt Based Graph Self-Supervised Learning

Haojie Li
School of Data Science, Qingdao
University of Science and Technology
Qingdao, China
lihaojie@qust.edu.cn

Mengjiao Zhang
School of Information Science and
Technology, Qingdao University of
Science and Technology
Qingdao, China
zhang_mj@mails.qust.edu.cn

Guanfeng Liu*
School of Computing, Macquarie
University
Sydney, Australia
guanfeng.liu@mq.edu.au

Qiang Hu
School of Information Science and
Technology, Qingdao University of
Science and Technology
Qingdao, China
huqiang@qust.edu.cn

Yan Wang
School of Computing, Macquarie
University
Sydney, Australia
yan.wang@mq.edu.au

Junwei Du*
School of Data Science, Qingdao
University of Science and Technology
Qingdao, China
dujunwei@qust.edu.cn

Abstract

Graph self-supervised learning (GSSL) alleviates the graph data labeling bottleneck without supervision, enabling wide application in domains like recommendation systems and social network analysis. High-frequency signals are valuable in GSSL for capturing local structural preferences, thereby enriching graph representations and boosting model performance. However, in practical applications, two critical problems hinder the efficient and robust use of these signals. First, the locality of high-frequency signals limits their full utilization by the model. Second, over-reliance on specific high-frequency signals will affect the model's generalization. To address the above problems, we propose the Frequency-Corrupt Based Graph Self-Supervised Learning (FC-GSSL) algorithm. Specifically, we generate corrupted graphs biased toward high-frequency signals by corrupting nodes and edges according to their low-frequency contributions. These corrupted graphs are fed as input to an autoencoder, with low-frequency and general features serving as the supervision. This compels the model to effectively fuse high- and low-frequency signals, thereby integrating and utilizing more valuable high-frequency information. Additionally, we design multiple sampling strategies and form diverse corrupted graphs based on the intersections and union between the results obtained from these strategies. By aligning the node representations from these views, the model can identify valuable frequency combinations, which helps reduce the negative impact of specific high-frequency components and improve generalization. FC-GSSL optimizes the design of GSSL for web applications, significantly improving model performance on complex web-related graphs, such as social networks and citation networks. This work makes a direct contribution to advancing the "Graph Algorithms and Modeling for the Web" research track. Experimental results on 14 datasets across multiple tasks illustrate the superiority of the proposed approach.

*Corresponding authors.



This work is licensed under a Creative Commons Attribution 4.0 International License.
WWW '26, Dubai, United Arab Emirates
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2307-0/2026/04
<https://doi.org/10.1145/3774904.3792492>

CCS Concepts

• Information systems → Data mining.

Keywords

Graph Neural Networks, Self-supervised Learning, Graph Autoencoders, Low-frequency Signal Analysis

ACM Reference Format:

Haojie Li, Mengjiao Zhang, Guanfeng Liu, Qiang Hu, Yan Wang, and Junwei Du. 2026. Frequency-Corrupt Based Graph Self-Supervised Learning. In *Proceedings of the ACM Web Conference 2026 (WWW '26)*, April 13–17, 2026, Dubai, United Arab Emirates. ACM, New York, NY, USA, 11 pages. <https://doi.org/10.1145/3774904.3792492>

1 Introduction

GSSL is broadly categorized into two paradigms: contrastive learning and generative learning [53]. Contrastive learning methods capture structural patterns by maximizing mutual information between positive pairs in different views [11, 21]. Meanwhile, generative learning methods learn structural details by reconstructing corrupted parts of the graph [25, 30]. GSSL can learn expressive graph representations without manual labels to alleviate the annotation bottleneck, and is widely applied in recommendation systems [21, 22], social network analysis [40, 57], and drug discovery [4, 17].

However, most GSSL methods [14, 50] still prefer low-frequency signals and overlook valuable high-frequency patterns, limiting the learning of expressive structural information [30]. To address this, recent studies focus on capturing high-frequency signals in graphs: for example, ChebyCF [18] and PolyCF [37] apply graph spectral filtering to enhance high-frequency sensitivity, while GraphPAE [30] and HOPE-WavePE [34] employ positional encodings to capture them effectively. Despite progress [30, 37], two critical problems hinder the efficient and robust use of these signals.

Problem 1. The locality of high-frequency signals limits their full utilization by the model. Current deep learning models are mainly based on statistical inductive learning [51]. They extract stable patterns from the overall data distribution to obtain generalizable and transferable representations [20]. However, in graph data, high-frequency components often reflect local preferences [24, 65]. These signals are sparse, non-stationary, and highly sensitive to sampling

and noise [58]. They lack statistical stability and fail to form consistent and representative patterns at the group level. Therefore, a fundamental conflict exists between the locality of high-frequency signals and statistical inductive learning. In conventional training procedures, models tend to preserve smooth and globally consistent patterns, potentially misclassifying valuable high-frequency information as noise and suppressing it. This results in insufficient utilization of high-order signals, thereby further compromising the model's representational capacity.

Problem 2. Over-reliance on specific high-frequency signals will affect the model's generalization. High-frequency signals in graph data, which often reflect local preferences and specific patterns [65], contradict the model's inherent need for smoothness and thus reduce its generalization ability [37]. Although some studies employ ideal low-pass filters to enhance the model's ability to extract low-frequency components, these methods merely combine high- and low-frequency signals through simple aggregation [5, 39]. They fail to effectively coordinate the relationships between signals of multiple frequencies, making it difficult to discern whether specific combinations of frequency signal patterns represent valid information or mere coincidental noise. Therefore, the model may still have excessive reliance on specific high-frequency signals. During training, the model may focus too much on these local details, leading to overfitting and thus affecting the model's generalization ability.

To address the above problems, we propose the **Frequency-Corrupt Based Graph Self-Supervised Learning (FC-GSSL)** algorithm. Specifically, we analyze the contribution of low-frequency signals in nodes and edges to the graph structure, from a spectral perspective [2, 3]. Based on this, we corrupt nodes and edges according to their low-frequency contributions, creating corrupted graphs biased toward high-frequency signals. These graphs are fed into an autoencoder under a corruption-reconstruction paradigm, where supervision relies on both low-frequency and general features. This compels the model to learn how to transform high-frequency signals into lower-frequency representations and to more effectively explore their collaborative relationships. Consequently, the model shifts its focus from merely extracting specific frequency signals to effectively fusing multi-frequency signals. This mechanism enhances the integration of valuable high-frequency signals, mitigating their underutilization. Additionally, we design multiple sampling strategies and form diverse corrupted graphs based on intersections and unions among their outcomes. Through aligning node representations from corrupted graphs, the model improves its ability to distinguish meaningful multi-frequency signals from random noise. This reduces the negative influence of specific high-frequency components, enhancing the model's robustness and generalization. To sum up, the contributions of our work are summarized as follows:

- We construct high-frequency-biased graphs as autoencoder inputs and set low-frequency and general features as reconstruction targets, compelling the model to fuse multi-frequency signals and better utilize valuable high-frequency information.
- We generate diverse corrupted graphs via multiple sampling strategies and align their node representations, enabling the model to identify valuable frequency combinations, mitigate specific high-frequency effects, and enhance generalization.

- We compare FC-GSSL with state-of-the-art baseline methods across multiple tasks, including node classification, graph property prediction, and transfer learning. Results show FC-GSSL consistently outperforms the baseline methods on 14 graphs, with higher accuracy and better generalization in most tasks.

2 Related Work

2.1 Graph Self-supervised Learning

Graph Contrastive Learning. Graph contrastive learning enhances model robustness through the alignment of positive sample pairs from different views. This process captures semantic invariance and reduces irrelevant information in node representations [11, 21]. For examples: SGL [54] and AutoGCL [59] achieve semantic-invariant learning by employing data augmentation techniques, such as node dropping and edge perturbation, and aligning representations of the same node across different views. MSSGCL [31] and GraphCL [61] leverage subgraph sampling and multi-scale contrastive learning to achieve fine-grained representation of node semantics. NCL [26], AdaCML [64], and IPCCF [21] leverage contrastive learning to align nodes with their neighbors in user-item graphs, generating better representations for recommendation.

Graph Generative Learning. Graph generative learning methods, such as graph autoencoders, require the model to reconstruct corrupted parts by leveraging available context, which facilitates the learning of intricate structural details within the graph [25, 30]. For examples: VGAE [19] and GATE [38] achieve self-supervised learning by corrupting and reconstructing links and features, respectively. GraphMAE [14] and GraphMAE2 [13], as masked graph autoencoders, address the issue of excessive reliance on neighboring information in traditional graph autoencoders through efficient feature masking and reconstruction, thereby improving the performance of generative models on various downstream tasks. HGMAE [45] and GraMI [63], using node attribute corruption and reconstruction, effectively encode complex heterogeneous graphs.

While above methods have shown promise, they mainly focus on low-frequency signals and neglect local preference patterns, which limits the ability to capture expressive structural information [30].

2.2 Usage of High-Frequency Signals

Graph Spectral Filtering. Graph spectral filtering employs effective spectral filters to extract high-frequency signals, thereby capturing local information in graphs [29, 37]. For examples: DSF [10] introduces a diversified spectral filtering framework that learns node-specific filter weights, thereby leveraging distinct local structural features. Sp²GCL [1] fuses spatial and spectral graph views through consistency capture between GNN-encoded spatial information and robust spectral features. BernNet [12], JacobiConv [52], and ChebyCF [18] leverage high-order polynomial graph filters to express arbitrarily smooth functions, thereby achieving effective extraction of diverse graph frequency signals. GF-CF [39] and PGSP [28] enhance the ability to capture frequency signals by integrating graph filter properties into collaborative filtering.

Positional Encodings. Infusing position information associated with high-frequency signals during graph encoding can enhance the model's capacity to capture local graph details, thus boosting its representational power [30]. For examples: Some studies

[7, 8] use the eigenvectors of the graph Laplacian matrix as initial node positional encodings to represent relative node relationships in the spectral domain. RFP [9] enhances model expressiveness by using random feature propagation instead of directly computing Laplacian eigenvectors, which helps preserve high-frequency components. SPE [16] perturbs the Laplacian matrix to generate robust positional encodings for enhanced high-frequency signal extraction. PEG [49] uses dual channels for updates of independent nodes and positional characteristics, with bidirectional constraints to resolve ambiguities of the sign and basis to improve high-frequency use.

Despite their success with high-frequency signals, the above methods are limited by signal locality and an over-reliance on specific signals, which hinders their overall effectiveness.

3 Preliminaries

In this section, we first define the relevant notations and basic concepts to facilitate the elaboration of our subsequent method.

Problem Definition. Given a graph $\mathcal{G} = \{\mathcal{V}, \mathcal{E}, \mathbf{X}\}$, where $\mathcal{V}$ denotes the node set with $\mathcal{V} = \{v_i\}_{i=1}^N$, $\mathcal{E} \subseteq \mathcal{V} \times \mathcal{V}$ denotes the edge set with $e_{ij} \in \mathcal{E}$ connecting node v_i and node v_j , and $\mathbf{X} \in \mathbb{R}^{N \times d}$ is the node feature matrix with d -dimensional feature vectors. The graph topology is represented by an adjacency matrix $\mathbf{A} \in \{0, 1\}^{N \times N}$, where $A_{ij} = 1$ if an edge exists between nodes v_i and v_j , and 0 otherwise. The objective of our work is to learn an unsupervised graph encoder $f_{enc}(\cdot)$ that maps the input graph to meaningful node representations $\mathbf{H} = f_{enc}(\mathcal{G}) \in \mathbb{R}^{N \times d_h}$ without using labeled data.

Laplacian Matrix, Eigenvalues and Eigenvectors. The normalized graph Laplacian is defined as $\mathbf{L} = \mathbf{I}_N - \mathbf{D}^{-1/2} \mathbf{A} \mathbf{D}^{-1/2}$, where $\mathbf{I}_N$ is the identity matrix and $\mathbf{D}$ is the diagonal degree matrix constructed from the graph's adjacency matrix $\mathbf{A}$, with each diagonal entry D_{ii} equal to the degree of node v_i . The Laplacian matrix $\mathbf{L}$ admits an eigen-decomposition $\mathbf{L} = \mathbf{U} \mathbf{\Lambda} \mathbf{U}^T$, where $\mathbf{U} = [\mathbf{u}_1, \mathbf{u}_2, \dots, \mathbf{u}_N]$ is an orthogonal matrix whose columns are the eigenvectors of $\mathbf{L}$, and $\mathbf{\Lambda} = \text{diag}(\lambda_1, \lambda_2, \dots, \lambda_N)$ is the diagonal matrix of the corresponding eigenvalues. These satisfy the relation $\mathbf{L} \mathbf{u}_i = \lambda_i \mathbf{u}_i$ for each i .

4 Methodology

4.1 Overview

The overall framework of FC-GSSL, as shown in Fig. 1, consists of four modules:

- **Low-Frequency Signal Contribution Module:** This module analyzes low-frequency signal contributions via the Laplacian matrix, generating high-frequency biased corrupted graphs.
- **Data Corruption Module:** This module develops and combines sampling strategies from low-frequency signal contributions for generating diverse high-frequency biased corrupted graphs.
- **FC-GSSL Encoder Module:** This module processes corrupted graphs through the encoder and aligns the resulting multi-view node representations to mitigate over-reliance on specific frequency signals and enhance generalization.
- **FC-GSSL Decoder Module:** This module decodes node representations and reconstructs corrupted data from both node features and edge structure, enabling the model to learn collaborative multi-frequency signal relationships for effective fusion.

4.2 Low-Frequency Signal Contribution

To enhance the model's use of high-frequency signals, we analyze low-frequency contributions in nodes and edges to generate corrupted graphs biased toward high-frequency information.

Analysis of Low-Frequency Signal Contribution in Edges. To analyze the contribution of low-frequency signals to edges, we propose a novel metric named the area under the curve of low-frequency signal contribution in edges (C_E), which is inspired by the AUC concept [23]. This metric aggregates signal contributions across all frequencies using the following formula:

$$C_{E,ij} = \frac{1}{K} \sum_{m=1}^K \frac{\sum_{n=1}^m |\mathbf{u}_{ni} \times \lambda_n \times \mathbf{u}_{nj}|}{\sum_{n=1}^K |\mathbf{u}_{ni} \times \lambda_n \times \mathbf{u}_{nj}|}, \quad (1)$$

where $C_{E,ij}$ represents the contribution value of low-frequency signals in edge e_{ij} to the graph structure, K denotes the number of top-frequency components we are concerned with, $\mathbf{u}_{ni}$ denotes the i -th element in eigenvector $\mathbf{u}_n$, and λ_n represents the n -th eigenvalue.

Analysis of Low-Frequency Signal Contribution in Nodes. After obtaining low-frequency signal contributions in edges, we aggregate them around each node to compute the low-frequency contribution in nodes C_N , as follows:

$$C_{N,i} = \frac{1}{|\mathcal{N}(i)|} \sum_{j \in \mathcal{N}(i)} C_{E,ij} \quad (2)$$

where $C_{N,i}$ represents the contribution value of low-frequency signals in node v_i to the graph structure, and $\mathcal{N}(i)$ represents the set of neighboring indices of node v_i .

4.3 Data Corruption

In this paper, we corrupt the original graph data based on low-frequency signal contributions to obtain corrupted graphs biased towards high-frequency signals. The process consists of two steps: corrupt item selection and corrupt target generation.

4.3.1 Corrupt Item Selection. In this section, we develop multiple sampling strategies based on the value and rank of low-frequency signal contributions. The final corruption targets are determined by combining the results from these strategies.

Value-based Sampling Strategy. In this strategy, we perform the sampling operation based on the contribution of low-frequency signals in nodes or edges to the graph structure, such that nodes or edges with higher contribution values have a higher probability of being sampled, which is defined as follows:

$$\mathcal{P} \sim \text{Multinomial}(C, T). \quad (3)$$

In this process, T items are drawn without replacement from the entire set, with the selection probability for each item proportional to its contribution value C . This is equivalent to sampling indices from a multinomial distribution. These sampled indices constitute the sampled item set $\mathcal{P}$. In practice, the value of T is determined by the sampling rate r .

Rank-based Sampling Strategy. To avoid over-reliance on the specific calculation method and numerical values of signal contributions, we designed a rank-based sampling strategy alongside the value-based approach. Specifically, we sort the contribution values by their magnitude and replace the original values with their

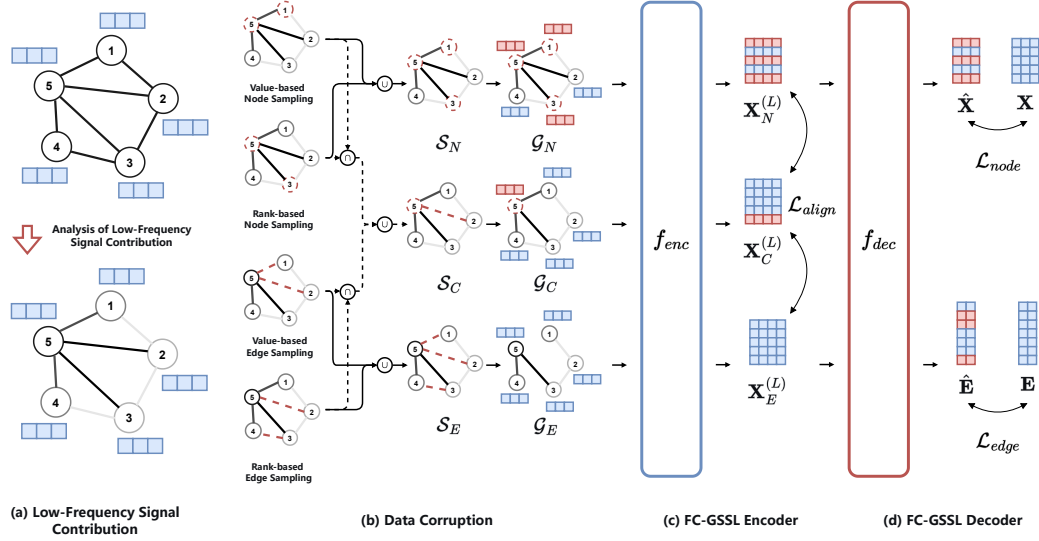


Figure 1: An overview of FC-GSSL

corresponding rank indices. The specific formula is as follows:

$$Q \sim \text{Multinomial}(RI(C), T), \quad (4)$$

where $RI(\cdot)$ is defined as the ranking and indexing operation, and Q is the sampled item set derived from the rank-based sampling strategy.

Corrupted Item Selection Strategy. To mitigate the model's over-reliance on specific high-frequency signals and the bias induced by specific sampling strategies, we draw on the Jaccard Similarity Coefficient [35], which asserts that the similarity between two sets increases with the ratio of the size of their intersection to the size of their union. Therefore, after generating sampled item sets based on value-based and rank-based sampling strategies, we aim for the data corruption caused by the intersection and union of these sets to have approximately the same impact on the model representations. This enables the model to integrate high-frequency signals while mitigating its over-reliance on specific high-frequency signals. Based on the above analysis, we construct the sampled item sets $S_N = \mathcal{P}_N \cup Q_N$, $S_{\tilde{N}} = \mathcal{P}_N \cap Q_N$, $S_E = \mathcal{P}_E \cup Q_E$, and $S_{\tilde{E}} = \mathcal{P}_E \cap Q_E$ with different sampling strategies, considering the contributions of low-frequency signals from nodes (C_N) and edges (C_E) to the graph structure. Specifically, $\mathcal{P}_N$ and Q_N represent the sampled item sets derived from C_N using the value-based and rank-based sampling strategies, respectively. Correspondingly, the sampling rates for nodes and edges are r_N and r_E , respectively. Furthermore, we design a comprehensive sampled item set $S_C = S_{\tilde{N}} \cup S_{\tilde{E}}$. By ensuring that the data corruption effects on the model are similar between sampled item sets S_N and S_C and between sampled item sets S_E and S_C , this method reduces the model's over-reliance on specific high-frequency signals while enhancing its stability.

4.3.2 Corruption Target Generation. After obtaining the sampled item sets, we perform data corruption operations based on the types of items in the sets to generate the corrupted graphs.

For node-type items, we perform feature masking. We follow GraphMAE [14] by replacing the features of the sampled nodes $\tilde{\mathcal{V}} = \mathcal{S} \cap \mathcal{V}$ with a learnable vector, where $\mathcal{S}$ represents the sampled item set on which we operate and $\mathcal{V}$ represents the original node set. The corrupted feature matrix $\tilde{\mathbf{X}}$ is defined as follows:

$$\tilde{\mathbf{X}}_i = \begin{cases} \mathbf{x}_{[M]}, & \text{if } v_i \in \tilde{\mathcal{V}} \\ \mathbf{X}_i, & \text{if } v_i \notin \tilde{\mathcal{V}} \end{cases} \quad (5)$$

where $\mathbf{X}_i \in \mathbb{R}^d$ denotes the original feature of node v_i , and $\mathbf{x}_{[M]} \in \mathbb{R}^d$ denotes a learnable vector.

For edge-type items, we perform edge dropping. We obtain the corrupted edge set $\tilde{\mathcal{E}}$ by the formula $\tilde{\mathcal{E}} = \mathcal{E} \setminus \mathcal{S}$, where $\mathcal{S}$ represents the sampled item set on which we operate, and $\mathcal{E}$ represents the original edge set.

After introducing the specific data corruption methods, we corrupted the sampled item sets S_N , S_E , and S_C , yielding the corresponding corrupted graphs $\mathcal{G}_N$, $\mathcal{G}_E$, and $\mathcal{G}_C$.

4.4 FC-GSSL Encoder

In this section, we detail the encoding and representation alignment process of FC-GSSL. In the encoding phase, we follow the GraphPAE [30] to integrate node features and pairwise distances via an attention mechanism, allowing effective fusion of multi-frequency signals, which can be formulated as:

$$\mathbf{X}^{(l+1)}, \mathbf{P}^{(l+1)} = f_{enc}^{(l+1)}(\mathbf{X}^{(l)}, \mathbf{P}^{(l)}), \quad (6)$$

where l denotes the encoder layer, ranging from 0 to L , and $f_{enc}^{(l+1)}$ denotes the process where the vector representation of layer l generates that of layer $l+1$ via message passing. $\mathbf{X}^{(l)} \in \mathbb{R}^{d_h}$ is the node feature matrix at the l -th layer, where its initial form $\mathbf{X}^{(0)} \in \mathbb{R}^{d_h}$ is derived by mapping the original node feature matrix $\mathbf{X}$ to a d_h -dimensional space using a multi-layer perceptron. $\mathbf{P}^{(l)} \in \mathbb{R}^{d_h}$ denotes the node relative positional representation at the l -th layer. Its initial form $\mathbf{P}^{(0)} \in \mathbb{R}^{d_h}$ is derived from the node relative position

matrix $\mathbf{P} \in \mathbb{R}^{N \times N}$, which is defined as:

$$\mathbf{P}_{ij} = \begin{cases} \|\mathbf{U}_i - \mathbf{U}_j\|_2, & \text{if } \mathbf{A}_{ij} = 1, \\ 0, & \text{otherwise.} \end{cases} \quad (7)$$

We transform matrix $\mathbf{P}$ into the representation $\mathbf{P}^{(0)}$ through a series of Gaussian RBF kernels, using the operation

$$\mathbf{P}_{ij}^{(0)} = \text{MLP}([G(\mathbf{P}_{ij}; \mu_1, \sigma), \dots, G(\mathbf{P}_{ij}; \mu_d, \sigma)]), \quad (8)$$

where MLP is another multilayer perceptron and $G(\mathbf{P}_{ij}; \mu_k, \sigma) = \exp(-(\mathbf{P}_{ij} - \mu_k)^2 / 2\sigma^2)$ is the k -th Gaussian basis function, parameterized by a mean of μ_k and a standard deviation of σ .

Message Passing Process. For the feature objective, we define the feature message passing path as

$$\begin{aligned} \mathbf{X}_i^{(l+1)} &= \sum_{j \in \mathcal{N}(i)} (\alpha_{ij}^{(l)} + \mathbf{P}_{ij}^{(l)}) \odot \mathbf{X}_j^{(l)}, \\ \alpha_{ij}^{(l)} &= f_{att}(\mathbf{X}_i^{(l)}, \mathbf{X}_j^{(l)}), \alpha_{ij}^{(l)} \in \mathbb{R}^{d_h}, \end{aligned} \quad (9)$$

where f_{att} is the attention function that calculates weights between neighboring nodes, and $\odot$ represents an element-wise multiplication. The specific implementation of this attention mechanism varies with the underlying graph encoder architecture. When using a GAT [47] as encoder, the parameter d_h signifies the number of attention heads, and the corresponding function is defined as

$$\alpha_{ij}^{(l)} = \text{LeakyReLU}(\mathbf{w}^T [\mathbf{W}^{(l)} \mathbf{X}_i^{(l)} \parallel \mathbf{W}^{(l)} \mathbf{X}_j^{(l)}]), \mathbf{w} \in \mathbb{R}^{2d_h}. \quad (10)$$

In contrast, when the GatedGCN architecture is used, d_h is assigned a value equal to the dimensionality of the node embeddings, and the functional relationship is defined as

$$\alpha_{ij}^{(l)} = \text{Sigmoid}(\mathbf{W}^{(l)} \mathbf{X}_i^{(l)} + \mathbf{W}^{(l)} \mathbf{X}_j^{(l)}). \quad (11)$$

This path takes into account the weights of both features and positions during the message passing process, allowing for a more refined fusion of multi-dimensional frequency signals.

For the position objective, we define the position message passing path as

$$\mathbf{P}_{ij}^{(l+1)} = \mathbf{P}_{ij}^{(l)} + \alpha_{ij}^{(l)}. \quad (12)$$

We use attention weights to continuously optimize positional representations by integrating more feature information. This process allows the positional representations to better approximate the actual node positions, thereby improving the accuracy of the model representations.

Generation and Alignment of Node Representations. After detailing the implementation of FC-GSSL, we use $\mathcal{G}_N$, $\mathcal{G}_E$, and $\mathcal{G}_C$ as inputs to the encoder to obtain the L -layer vector representations $\mathbf{X}_N^{(L)}$, $\mathbf{X}_E^{(L)}$, and $\mathbf{X}_C^{(L)}$ of the graph nodes, which serve as the final node embeddings, respectively. To align node representations between different corrupted graphs, a contrastive objective based on InfoNCE loss [6] is employed. This objective functions by maximizing the similarity between representations of the same node from different corrupted graphs while minimizing the similarity between those of dissimilar nodes, formulated as:

$$\mathcal{I}(\mathbf{X}', \mathbf{X}'') = \frac{1}{N} \sum_{i=0}^N -\log \frac{\exp(s(\mathbf{x}'_i, \mathbf{x}''_i)/\tau)}{\sum_{i'=0}^N \exp(s(\mathbf{x}'_i, \mathbf{x}''_{i'})/\tau)}, \quad (13)$$

where $\mathbf{x}'_i$ denotes the representation of the i -th node in the feature matrix $\mathbf{X}'$, $s(\cdot, \cdot)$ indicates the similarity function between nodes,

and τ represents the temperature parameter. By aligning the node vector representations $\mathbf{X}_N^{(L)}$ with $\mathbf{X}_C^{(L)}$, and $\mathbf{X}_E^{(L)}$ with $\mathbf{X}_C^{(L)}$, we force the model to avoid over-reliance on specific high-frequency signals. The loss function is defined as follows:

$$\mathcal{L}_{align} = \mathcal{I}(\mathbf{X}_N^{(L)}, \mathbf{X}_C^{(L)}) + \mathcal{I}(\mathbf{X}_E^{(L)}, \mathbf{X}_C^{(L)}). \quad (14)$$

4.5 FC-GSSL Decoder

Depending on the type of corruption operation, we reconstruct the corrupted information from two sources: the feature of the corrupted nodes and the structure of the corrupted edges.

Reconstructing the Features of Corrupted Nodes. For the corrupted graph $\mathcal{G}_N$, we recover the corrupted information by reconstructing the node features, where $\mathcal{S}_N$ denotes the set of corrupted nodes, $\mathbf{X}$ denotes the original node feature matrix, and $\mathbf{X}_N^{(L)}$ denotes for the encoded node representations. Employing a feature decoder, we map the node representations back to the original feature space. The process is as follows:

$$\hat{\mathbf{X}} = f_{dec}^N(\mathbf{X}_N^{(L)}), \quad (15)$$

where $\hat{\mathbf{X}}$ represents the reconstructed feature matrix. Following GraphMAE and GraphPAE, we adopt the scaled cosine error (SCE) as the feature reconstruction loss

$$\mathcal{L}_{node} = \frac{1}{|\mathcal{S}_N|} \sum_{v_i \in \mathcal{S}_N} \left(1 - \frac{\mathbf{X}_i^T \hat{\mathbf{X}}_i}{\|\mathbf{X}_i\| \cdot \|\hat{\mathbf{X}}_i\|}\right)^\gamma, \quad (16)$$

where $\gamma \geq 1$ is a hyperparameter.

Reconstructing the Structure of Corrupted Edges. For the corrupted graph $\mathcal{G}_E$, we recover the corrupted information by reconstructing the edge feature representations associated with its edges, where $\mathcal{S}_E$ denotes the set of corrupted edges, and $\mathbf{X}_E^{(L)}$ represents the encoded node representations. For an edge e_{ij} , its original edge feature can be denoted as $\mathbf{E}_{ij} = \mathbf{U}_i \odot \mathbf{U}_j$, and its representation derived from the learned node embeddings can be denoted as $\tilde{\mathbf{E}}_{ij} = \mathbf{X}_{E,i}^{(L)} \odot \mathbf{X}_{E,j}^{(L)}$. We also construct a decoder to achieve the transformation of edge features, with the process as follows:

$$\hat{\mathbf{E}} = f_{dec}^E(\tilde{\mathbf{E}}), \quad (17)$$

where $\hat{\mathbf{E}}$ and $\mathbf{E}$ share the same feature dimension. Then, the edge structure reconstruction loss is defined as follows:

$$\mathcal{L}_{edge} = \frac{1}{|\mathcal{S}_E|} \sum_{e_{ij} \in \mathcal{S}_E} \left(1 - \frac{\mathbf{E}_{ij}^T \hat{\mathbf{E}}_{ij}}{\|\mathbf{E}_{ij}\| \cdot \|\hat{\mathbf{E}}_{ij}\|}\right)^\gamma. \quad (18)$$

Having described all the components of FC-GSSL, its overall loss can be formulated as a weighted combination of multiple losses

$$\mathcal{L} = \mathcal{L}_{node} + \alpha \mathcal{L}_{edge} + \beta \mathcal{L}_{align}, \quad (19)$$

where α and β are hyperparameters. Detailed training procedure for FC-GSSL is provided in **Appendix A.1**.

4.6 Model Analysis

4.6.1 Analysis of the Design Rationale of FC-GSSL. GSSL integrates two paradigms: contrastive learning, which aligns positive samples across views to enhance robustness [11, 21], and generative learning, which reconstructs corrupted structures to capture graph details [25, 30]. Based on these premises, we propose the FC-GSSL model with the following design:

- We introduce a corrupted graph with high-frequency bias into an autoencoder, supervising the decoder with low-frequency and general features. This design forces the model to learn structural details and transform high-frequency signals into low-frequency representations, thereby improving multi-frequency fusion, enhancing cross-frequency collaborative understanding, and strengthening high-frequency information utilization.
- We generate multiple corrupted graphs through varied sampling strategies, using intersections and unions of their results. By aligning node representations across these graphs, we encourage the model to learn invariant semantic information, thereby enabling effective discrimination between high-frequency signals and noise. This reduces overreliance on specific frequency signals and improves the model’s generalization capability.

4.6.2 Comparison with Existing Methods. In this section, we compare FC-GSSL with existing methods that utilize high-frequency signals to evaluate the effectiveness and advantages of our model in leveraging such signals.

- **Comparison with Existing Spectral Filtering Methods.** Compared with methods PolyCF [37] and Sp²GCL [1], our focus shifts from designing effective spectral filters to extract high-frequency signals, to developing a mechanism that forces the model to learn collaborative relationships among different frequency signals. By facilitating effective aggregation, this mechanism mitigates the conflict between the locality of high-frequency signals and statistical inductive learning, thereby enhancing their utilization.
- **Comparison with Existing Positional Encoding Methods.** Compared with methods GraphPAE [30] and HOPE-WavePE [34], we construct corrupted graphs biased toward high-frequency components and guide the model in learning the transformation to low-frequency and general features, thus improving positional encoding efficiency and promoting frequency-domain feature fusion. In addition, contrastive learning is introduced to reduce reliance on specific frequencies and enhance generalization.

The time complexity analysis is provided in **Appendix A.2**.

5 Experiments

In this section, we conduct extensive experiments on node classification, graph prediction, and transfer learning to evaluate FC-GSSL. Dataset statistics and key model parameters are provided in **Appendices B.1** and **B.2**, respectively. Detailed hyperparameter configurations for replication are available in the source code at <https://github.com/rookitkitlee/FC-GSSL>.

5.1 Node Classification

Settings. To validate the performance of the FC-GSSL model on the node classification task, we conducted experiments using accuracy as the evaluation metric on 6 representative heterogeneous graphs: BlogCatalog [33], Chameleon, Squirrel, Actor [36], arXiv-year [15], and Penn94 [46]. Of these, arXiv-year and Penn94 are large-scale graphs with node sizes exceeding 40,000, thus effectively assessing the scalability of the proposed method.

For a comprehensive evaluation, FC-GSSL is compared against a wide range of representative graph self-supervised learning methods. These methods are broadly divided into two categories: **graph**

contrastive learning methods, such as DGI [48], BGRL [44], MVGRL [11], CCA-SSG [62], and Sp²GCL [1]; and **graph autoencoders**, including VGAE [19], GraphMAE [14], GraphMAE2 [13], MaskGAE [25], S2GAE [43], AUG-MAE [50], and GraphPAE [30]. Of these, Sp²GCL and GraphPAE extract high-frequency signals in graphs through graph spectral filtering and positional encodings, respectively. Using them as baselines can better demonstrate the effectiveness of our model in integrating high-frequency signals.

For the node classification task, we employ a GAT encoder with 4 attention heads, 1024 hidden units per layer, and the number of layers selected from {2, 3}. Node and edge features are decoded using two-layer MLPs with ReLU activations. For the downstream node classification task, the encoder weights are frozen, and a linear classifier is trained on the generated node embeddings. All models are optimized with Adam, and we report the average results over 5 experimental runs per graph.

Results. Table 1 shows the experimental results. From it we observe that: (1) Models Sp²GCL and GraphPAE, which focus on high-frequency signals, have a clear advantage over other baseline models in the same category on most datasets; (2) FC-GSSL outperforms all baselines, achieving the best overall performance.

The reasons for these results are: (1) Models emphasizing high-frequency signals better capture local structure, enriching node representations and improving classification. (2) FC-GSSL enhances existing methods by addressing the underutilization of high-frequency signals and over-reliance on specific components. It uses high-frequency-biased corrupted graphs as autoencoder inputs, with low-frequency and general features as targets, forcing effective multi-band signal integration. Generating diverse corrupted graphs and aligning their representations reduces sensitivity to specific components, improving generalization. Consequently, FC-GSSL strengthens local feature extraction and achieves superior node classification performance.

5.2 Graph Prediction

Settings. The evaluation of FC-GSSL for graph-level tasks is conducted across 7 OGB datasets [15]. This benchmark includes 3 graph regression and 4 graph classification problems. In all experiments, the standard public splits are employed to guarantee a fair comparison. Model performance is measured by MSE for regression and ROC-AUC for classification.

Our evaluation on graph prediction tasks involved a comparative analysis with benchmark methods from two categories: **graph contrastive learning** methods, such as InfoGraph [42], GraphCL [61], MVGRL [11], JOAO [60], and Sp²GCL [1]; and **graph autoencoders**, including GraphMAE [14], GraphMAE2 [13], StructMAE [27], AUG-MAE [50], and GraphPAE [30].

For the graph prediction tasks, we employ a two-layer GatedGCN encoder (hidden dimension $d_h=300$) and two-layer MLP decoders. During evaluation, node representations from the frozen encoder are pooled to form graph representations, which are then fed into a linear classifier. The model is trained using the Adam optimizer, and the reported results represent the average and standard deviation over 5 experimental runs per graph.

Results. Table 2 presents the performance of the FC-GSSL model on graph prediction tasks. From the results, it can be observed

Table 1: Node classification results of different graph self-supervised learning, mean accuracy (%) $\pm$ standard deviation. Bold indicates the best performance and underline means the runner-up.

Dataset	Small Graphs				Large Graphs	
	BlogCatalog	Chameleon	Squirrel	Actor	arXiv-year	Penn94
DGI	72.07 $\pm$ 0.16	43.83 $\pm$ 0.14	34.56 $\pm$ 0.10	27.98 $\pm$ 0.09	-	-
BGRL	79.74 $\pm$ 0.46	61.24 $\pm$ 1.07	43.24 $\pm$ 0.52	26.61 $\pm$ 0.57	41.43 $\pm$ 0.04	63.31 $\pm$ 0.491
MVGRL	63.24 $\pm$ 0.94	73.19 $\pm$ 0.42	60.09 $\pm$ 0.44	34.64 $\pm$ 0.20	-	-
CCA-SSG	74.00 $\pm$ 0.28	75.00 $\pm$ 0.75	61.58 $\pm$ 1.98	27.79 $\pm$ 0.58	40.78 $\pm$ 0.01	62.63 $\pm$ 0.20
Sp ² GCL	72.73 $\pm$ 0.46	78.88 $\pm$ 1.04	62.61 $\pm$ 0.87	34.70 $\pm$ 0.92	39.09 $\pm$ 0.02	68.80 $\pm$ 0.45
VGAE	60.47 $\pm$ 1.84	62.32 $\pm$ 1.90	42.50 $\pm$ 1.35	31.57 $\pm$ 0.75	36.39 $\pm$ 0.21	55.31 $\pm$ 0.28
GraphMAE	79.90 $\pm$ 1.13	79.50 $\pm$ 0.57	61.13 $\pm$ 0.60	32.15 $\pm$ 1.33	40.30 $\pm$ 0.04	67.97 $\pm$ 0.21
GraphMAE2	77.34 $\pm$ 0.12	79.13 $\pm$ 0.19	70.27 $\pm$ 0.88	34.48 $\pm$ 0.26	38.97 $\pm$ 0.03	67.86 $\pm$ 0.42
MaskGAE	73.10 $\pm$ 0.08	74.50 $\pm$ 0.87	68.53 $\pm$ 0.44	33.44 $\pm$ 0.34	40.59 $\pm$ 0.04	63.84 $\pm$ 0.03
S2GAE	75.76 $\pm$ 0.43	60.24 $\pm$ 0.37	68.60 $\pm$ 0.56	26.17 $\pm$ 0.38	40.32 $\pm$ 0.12	70.24 $\pm$ 0.09
AUG-MAE	82.03 $\pm$ 0.69	70.10 $\pm$ 1.88	62.57 $\pm$ 0.67	33.42 $\pm$ 0.38	37.10 $\pm$ 0.13	69.90 $\pm$ 0.43
GraphPAE	85.76 $\pm$ 1.22	80.51 $\pm$ 1.25	72.05 $\pm$ 1.40	38.55 $\pm$ 1.35	41.85 $\pm$ 0.04	71.79 $\pm$ 0.37
FC-GSSL(Ours)	88.70$\pm$1.10	82.31$\pm$0.51	74.03$\pm$0.84	41.94$\pm$0.92	42.00$\pm$0.11	73.50$\pm$0.40

Table 2: Graph regression and classification results of different graph self-supervised learning on OGB datasets. Bold indicates the best performance and underline means the runner-up. $\downarrow$ means lower the better and $\uparrow$ means higher the better.

Task	Regression (Metric: RMSE $\downarrow$)			Classification (Metric: ROC-AUC% $\uparrow$)			
	molso	molipo	molreesolv	molbase	molbbbp	molclintox	moltox21
InfoGraph	1.344 $\pm$ 0.178	1.005 $\pm$ 0.023	10.005 $\pm$ 8.147	73.64 $\pm$ 3.64	66.33 $\pm$ 2.79	64.50 $\pm$ 5.32	69.74 $\pm$ 0.57
GraphCL	1.272 $\pm$ 0.089	0.910 $\pm$ 0.016	7.679 $\pm$ 2.748	73.32 $\pm$ 2.70	68.22 $\pm$ 2.19	74.92 $\pm$ 4.42	72.40 $\pm$ 1.07
MVGRL	1.433 $\pm$ 0.145	0.962 $\pm$ 0.036	9.024 $\pm$ 1.982	74.88 $\pm$ 1.43	67.24 $\pm$ 3.19	73.84 $\pm$ 2.75	70.48 $\pm$ 0.83
JOAO	1.285 $\pm$ 0.121	0.865 $\pm$ 0.032	5.131 $\pm$ 0.782	74.43 $\pm$ 1.94	67.62 $\pm$ 1.29	71.28 $\pm$ 4.12	71.38 $\pm$ 0.92
Sp ² GCL	1.235 $\pm$ 0.119	0.835 $\pm$ 0.026	4.144 $\pm$ 0.573	78.76 $\pm$ 1.43	<u>68.72$\pm$1.53</u>	80.88 $\pm$ 3.86	73.06 $\pm$ 0.75
GraphMAE	1.050 $\pm$ 0.034	0.850 $\pm$ 0.022	2.740 $\pm$ 0.233	79.14 $\pm$ 1.31	66.55 $\pm$ 1.78	80.56 $\pm$ 5.55	73.84 $\pm$ 0.58
GraphMAE2	1.225 $\pm$ 0.081	0.885 $\pm$ 0.019	2.913 $\pm$ 0.293	80.74 $\pm$ 1.53	67.67 $\pm$ 1.44	75.75 $\pm$ 3.65	72.93 $\pm$ 0.69
StructMAE	1.499 $\pm$ 0.043	1.089 $\pm$ 0.002	2.568 $\pm$ 0.262	77.75 $\pm$ 0.42	65.66 $\pm$ 1.16	79.42 $\pm$ 4.56	71.13 $\pm$ 0.61
AUG-MAE	1.248 $\pm$ 0.026	0.917 $\pm$ 0.013	2.395 $\pm$ 0.158	78.54 $\pm$ 2.49	67.05 $\pm$ 0.63	82.66 $\pm$ 1.98	74.33 $\pm$ 0.07
GraphPAE	<u>1.015$\pm$0.045</u>	<u>0.810$\pm$0.018</u>	<u>2.348$\pm$0.215</u>	<u>81.11$\pm$1.24</u>	68.56 $\pm$ 0.71	<u>82.69$\pm$3.39</u>	<u>74.46$\pm$0.54</u>
FC-GSSL(Ours)	1.001$\pm$0.042	0.789$\pm$0.014	2.279$\pm$0.233	82.69$\pm$0.81	68.92$\pm$1.73	83.40$\pm$1.67	74.61$\pm$0.80

Table 3: Quantum chemistry property results of transfer learning on QM9. The best and runner-up results are highlighted with bold and underline, respectively.

Task	μ	α	ϵ_{homo}	ϵ_{lumo}	$\Delta\epsilon$	R^2	ZPVE	U_0	U	H	G	C_o
Unit	D	a_0^3	10^{-2}meV	10^{-2}meV	10^{-2}meV	a_0^2	10^{-2}meV	meV	meV	meV	meV	cal/mol/K
GraphCL	1.035	2.321	2.030	3.667	4.523	40.725	2.063	2.461	1.745	1.734	1.751	1.747
GraphMAE	1.030	2.924	2.407	6.373	4.813	41.955	4.623	1.411	2.207	2.208	2.207	2.200
Mole-BERT	1.031	1.918	1.477	4.127	4.240	44.374	2.190	2.532	2.509	2.511	2.516	2.508
SimSGT	1.064	2.413	2.837	4.227	4.107	40.504	2.127	1.948	2.420	2.416	2.416	2.410
GraphPAE	0.703	<u>0.879</u>	<u>1.199</u>	<u>2.141</u>	<u>2.289</u>	36.480	0.502	<u>0.510</u>	<u>0.639</u>	<u>0.639</u>	<u>0.641</u>	<u>0.643</u>
FC-GSSL(Ours)	<u>0.709</u>	0.851	1.194	2.121	2.231	<u>36.659</u>	<u>0.577</u>	0.490	0.633	0.635	0.636	0.631

that the model achieves excellent performance on both graph regression and graph classification tasks, significantly outperforming existing baseline methods based on contrastive learning and graph autoencoders.

The reasons for these results are that graph prediction tasks typically involve structures of relatively small scale, which require more fine-grained extraction of local features. Compared to other baseline models, FC-GSSL places greater emphasis on utilizing effective high-frequency signals while filtering out ineffective ones, allowing it to capture local features in graphs more precisely and thereby improving the accuracy of graph prediction tasks.

5.3 Transfer Learning

Settings. We evaluated FC-GSSL’s generalization via transfer learning for molecular property prediction. The encoder was pre-trained on 2 million ZINC15 [41] molecules and fine-tuned on QM9 [55] to predict quantum properties. We compare it with GraphCL [61], GraphMAE [14], Mole-BERT [56], SimSGT [32], and GraphPAE [30]. The encoder uses a five-layer GatedGCN (hidden dim 300), followed by a two-layer MLP for prediction. Fine-tuning trains both the encoder and the MLP on labeled downstream task data. QM9 is split 8:1:1 into training/validation/test sets. Experiments use Adam optimizer and are repeated 5 times for average results.

Table 4: Ablation studies on key components of FC-GSSL.

Dataset	Node Classification ($\uparrow$)		Graph Regression ($\downarrow$)		Graph Classification ($\uparrow$)	
	Squirrel	Actor	molesol	molipo	molbase	molbbbp
w/o cn	73.24	40.88	1.022	0.798	79.35	67.72
w/o ce	72.48	41.52	1.025	0.805	80.09	67.01
w/o cne	71.93	40.58	1.028	0.816	78.80	66.50
w/o so	73.39	41.35	1.030	0.806	78.54	66.80
w/o sa	73.21	38.43	1.016	0.804	79.97	67.35
w/o soa	72.34	37.58	1.054	0.808	77.65	66.20
FC-GSSL	74.03	41.94	1.001	0.789	82.69	68.92

Results. Table 3 shows the transfer learning results. The FC-GSSL model outperforms baseline methods on most tasks. This is because it focuses not on extracting high-frequency signals, but on effectively fusing signals of different frequencies. We also use contrastive learning to reduce the model’s dependence on specific frequencies, improving its generalization. As a result, FC-GSSL can learn high-quality frequency patterns and achieves more reliable and transferable performance than other transfer learning methods.

5.4 Ablation Studies

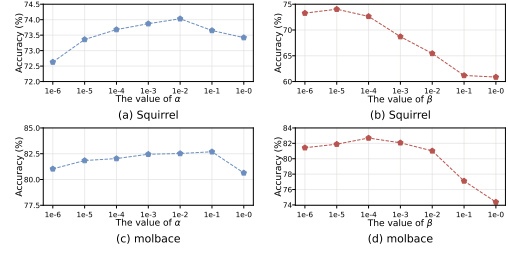
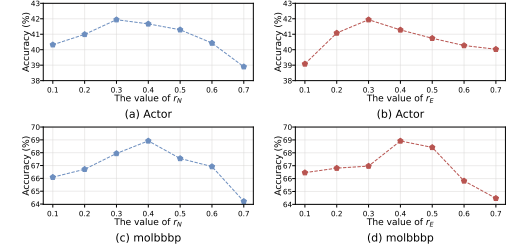
In this section, we validate the effectiveness of our design for generating and aligning corrupted graphs based on frequency contribution by employing variant experiments evaluated on 3 tasks across 6 datasets. (1) **w/o cn**: Eliminate corrupted nodes based on low-frequency signal contributions and adopt a random corruption strategy. (2) **w/o ce**: Eliminate corrupted edges based on low-frequency signal contributions and adopt a random corruption strategy. (3) **w/o cne**: Eliminate all operations corrupted by low-frequency signal contributions and adopt a random corruption strategy. (4) **w/o so**: Eliminate intersection and union operations across sampling strategies, using the value-based corrupted graph as input and the rank-based corrupted graph as the contrastive view. (5) **w/o sa**: Eliminate alignment between representations from different corrupted graphs. (6) **w/o soa**: Eliminate intersection and union operations across sampling strategies, and eliminate alignment between representations from different corrupted graphs.

Based on the ablation study results as shown in Table 4, we can draw the following conclusions: (1) Comparing FC-GSSL with its variants **w/o cn**, **w/o ce**, and **w/o cne** shows that using high-frequency corrupted graphs as autoencoder inputs, guided by low-frequency learning and general features, effectively directs the model’s frequency learning process and significantly improves high-frequency utilization. (2) Comparing FC-GSSL with its variants **w/o so**, **w/o sa**, and **w/o soa** shows that generating diverse corrupted graphs and aligning their node representations can suppress irrelevant frequency dependencies and reduce interference from unrelated signals, thereby enhancing model generalization.

5.5 Hyperparameter Analysis

We perform a thorough parameter analysis on the loss weights α and β , the sampling rates r_N and r_E . The experimental results are shown in Fig. 2 and Fig. 3.

The experimental results lead to the following conclusions: (1) As shown in Fig. 2, the change in loss weight α has less impact on performance than weight β . This is because the tasks of reconstructing the edge structure and node features are similar; consequently,

**Figure 2: The Influence of the loss weights α and β .****Figure 3: The Influence of the sampling rates r_N and r_E .**

the additional information provided by the $\mathcal{L}_{edge}$ term, which is controlled by α , is limited. Furthermore, edge corruption helps align representations from different corrupted graphs, enhancing the model’s sensitivity to the $\mathcal{L}_{align}$ term, which is controlled by β . (2) As shown in Fig. 3, the model is more sensitive to sampling rate changes on the molbbbp dataset than on the Actor dataset. This is because the limited scale of the molbbbp dataset restricts the model’s ability to learn from deeper neighbors, thereby making the sampling rate more critical.

The above experiments demonstrate that FC-GSSL is an effective graph self-supervised learning model. We conduct extensive experiments on 14 datasets across tasks including node classification, graph prediction, and transfer learning to validate the efficacy of our approach. Ablation studies further confirm that each component of the model contributes positively to performance improvement. Finally, we analyze key hyperparameters to provide deeper insights into their impact on the model’s performance.

6 Conclusion

In this paper, we have proposed the Frequency-Corrupt Based Graph Self-Supervised Learning (FC-GSSL) algorithm. Our approach constructed high-frequency-biased corrupted graphs as inputs to graph autoencoders, enhancing high-frequency information utilization. Additionally, we designed multiple sampling strategies to generate diverse corrupted graphs and aligned their node representations, preventing overreliance on specific high-frequency patterns and improving generalization capability. Experimental results from multiple tasks on 14 graph datasets have demonstrated the effectiveness of our proposed approach.

7 ACKNOWLEDGMENTS

This work is sponsored by the National Natural Science Foundation of China (Nos. 62506192, 62202253, 62272254, 62172249, 61973180), and the Natural Science Foundation of Shandong Province (Nos. ZR2024QF199, ZR2022MF326, ZR2021MF092, ZR2021QF074).

References

- [1] Deyu Bo, Yuan Fang, Yang Liu, and Chuan Shi. 2023. Graph contrastive learning with stable and scalable spectral encoding. *Advances in Neural Information Processing Systems* 36 (2023), 45516–45532.
- [2] Deyu Bo, Chuan Shi, Lele Wang, and Renjie Liao. 2023. Specformer: Spectral graph neural networks meet transformers. *arXiv preprint arXiv:2303.01028* (2023).
- [3] Deyu Bo, Xiao Wang, Yang Liu, Yuan Fang, Yawen Li, and Chuan Shi. 2023. A survey on spectral graph neural networks. *arXiv preprint arXiv:2302.05631* (2023).
- [4] Pietro Bongini, Monica Bianchini, and Franco Scarselli. 2021. Molecular generative graph neural networks for drug discovery. *Neurocomputing* 450 (2021), 242–252.
- [5] Xuheng Cai, Chao Huang, Lianghao Xia, and Xubin Ren. 2023. LightGCL: Simple yet effective graph contrastive learning for recommendation. *arXiv preprint arXiv:2302.08191* (2023).
- [6] Ting Chen, Simon Kornblith, Mohammad Norouzi, and Geoffrey Hinton. 2020. A simple framework for contrastive learning of visual representations. In *ICML*. 1597–1607.
- [7] Vijay Prakash Dwivedi, Chaitanya K Joshi, Anh Tuan Luu, Thomas Laurent, Yoshua Bengio, and Xavier Bresson. 2023. Benchmarking graph neural networks. *Journal of Machine Learning Research* 24, 43 (2023), 1–48.
- [8] Vijay Prakash Dwivedi, Anh Tuan Luu, Thomas Laurent, Yoshua Bengio, and Xavier Bresson. 2021. Graph neural networks with learnable structural and positional representations. *arXiv preprint arXiv:2110.07875* (2021).
- [9] Moshe Eliasof, Fabrizio Frasca, Beatrice Bevilacqua, Eran Treister, Gal Chechik, and Haggai Maron. 2023. Graph positional encoding via random feature propagation. In *ICML*. PMLR, 9202–9223.
- [10] Jingwei Guo, Kaizhu Huang, Xinpeng Yi, and Rui Zhang. 2023. Graph neural networks with diverse spectral filtering. In *WWW*. 306–316.
- [11] Kaveh Hassani and Amir Hosein Khasahmadi. 2020. Contrastive multi-view representation learning on graphs. In *ICML*. 4116–4126.
- [12] Mingguo He, Zhewei Wei, Hongteng Xu, et al. 2021. Bernnet: Learning arbitrary graph spectral filters via bernstein approximation. *NeurIPS* 34 (2021), 14239–14251.
- [13] Zhenyu Hou, Yufei He, Yukuo Cen, Xiao Liu, Yuxiao Dong, Evgeny Kharlamov, and Jie Tang. 2023. Graphmae2: A decoding-enhanced masked self-supervised graph learner. In *WWW*. 737–746.
- [14] Zhenyu Hou, Xiao Liu, Yukuo Cen, Yuxiao Dong, Hongxia Yang, Chunjie Wang, and Jie Tang. 2022. Graphmae: Self-supervised masked graph autoencoders. In *KDD*. 594–604.
- [15] Weihua Hu, Matthias Fey, Marinka Zitnik, Yuxiao Dong, Hongyu Ren, Bowen Liu, Michele Catasta, and Jure Leskovec. 2020. Open graph benchmark: Datasets for machine learning on graphs. *Advances in neural information processing systems* 33 (2020), 22118–22133.
- [16] Yanan Huang, William Lu, Joshua Robinson, Yu Yang, Muhan Zhang, Stefanie Jegelka, and Pan Li. 2023. On the stability of expressive positional encodings for graphs. *arXiv preprint arXiv:2310.02579* (2023).
- [17] Dejun Jiang, Zhenxing Wu, Chang-Yu Hsieh, Guangyong Chen, Ben Liao, Zhe Wang, Chao Shen, Dongsheng Cao, Jian Wu, and Tingjun Hou. 2021. Could graph neural networks learn better molecular representation for drug discovery? A comparison study of descriptor-based and graph-based models. *Journal of cheminformatics* 13, 1 (2021), 12.
- [18] Chanwoo Kim, Jinkyu Sung, Yebonn Han, and Joonseok Lee. 2025. Graph Spectral Filtering with Chebyshev Interpolation for Recommendation. In *SIGIR*. 1964–1974.
- [19] Thomas N Kipf and Max Welling. 2016. Variational graph auto-encoders. *arXiv preprint arXiv:1611.07308* (2016).
- [20] Yunwen Lei, Tianbao Yang, Yiming Ying, and Ding-Xuan Zhou. 2023. Generalization analysis for contrastive representation learning. In *ICML*. 19200–19227.
- [21] Haojie Li, Junwei Du, Guanfeng Liu, Feng Jiang, Yan Wang, and Xiaofang Zhou. 2025. Intent Propagation Contrastive Collaborative Filtering. *IEEE Transactions on Knowledge and Data Engineering* 01 (2025), 1–14.
- [22] Haojie Li, Guanfeng Liu, Qiang Hu, Yan Wang, Dunwei Gong, and Junwei Du. 2025. Candidate-aware graph prompt-tuning for recommendation. *Pattern Recognition* (2025), 111733.
- [23] Jing Li. 2024. Area under the ROC Curve has the most consistent evaluation for binary classification. *PLoS one* 19, 12 (2024), e0316019.
- [24] Jialu Li, Yu Wang, Pengfei Zhu, Wanyu Lin, and Qinghua Hu. 2024. What matters in graph class incremental learning? An information preservation perspective. *NeurIPS* 37 (2024), 26195–26223.
- [25] Jintang Li, Ruofan Wu, Wangbin Sun, Liang Chen, Sheng Tian, Liang Zhu, Changhua Meng, Zibin Zheng, and Weiqiang Wang. 2023. What's behind the mask: Understanding masked graph modeling for graph autoencoders. In *KDD*. 1268–1279.
- [26] Zihan Lin, Changxin Tian, Yupeng Hou, and Wayne Xin Zhao. 2022. Improving graph collaborative filtering with neighborhood-enriched contrastive learning. In *WWW*. 2320–2329.
- [27] Chuang Liu, Yuyao Wang, Yibing Zhan, Xueqi Ma, Dapeng Tao, Jia Wu, and Wenbin Hu. 2024. Where to mask: structure-guided masking for graph masked autoencoders. In *IJCAI*. Association for Computing Machinery (ACM), 2180–2188.
- [28] Jiahao Liu, Dongsheng Li, Hansu Gu, Tun Lu, Peng Zhang, Li Shang, and Ning Gu. 2023. Personalized graph signal processing for collaborative filtering. In *WWW*. 1264–1272.
- [29] Nian Liu, Xiao Wang, Deyu Bo, Chuan Shi, and Jian Pei. 2022. Revisiting graph contrastive learning from the perspective of graph spectrum. *NeurIPS* 35 (2022), 2972–2983.
- [30] Yang Liu, Deyu Bo, Wenxuan Cao, Yuan Fang, Yawen Li, and Chuan Shi. 2025. Graph Positional Autoencoders as Self-supervised Learners. In *KDD*. 1867–1878.
- [31] Yanbei Liu, Yu Zhao, Xiao Wang, Lei Geng, and Zhitao Xiao. 2024. Multi-scale subgraph contrastive learning. *arXiv preprint arXiv:2403.02719* (2024).
- [32] Zhiyuan Liu, Yaorui Shi, An Zhang, Enzhi Zhang, Kenji Kawaguchi, Xiang Wang, and Tat-Seng Chua. 2023. Rethinking tokenizer and decoder in masked graph modeling for molecules. *NeurIPS* 36, 25854–25875.
- [33] Zaiqiao Meng, Shangsong Liang, Hongyan Bao, and Xiangliang Zhang. 2019. Co-embedding attributed networks. In *WSDM*. 393–401.
- [34] Viet Anh Nguyen, Nhat Khang Ngo, and Truong Son Hy. 2024. Range-aware Positional Encoding via High-order Pretraining: Theory and Practice. *arXiv preprint arXiv:2409.19117* (2024).
- [35] Soon Hyeok Park and Kyoungok Kim. 2023. Collaborative filtering recommendation system based on improved Jaccard similarity. *Journal of Ambient Intelligence and Humanized Computing* 14, 8 (2023), 11319–11336.
- [36] Hongbin Pei, Bingzhe Wei, Kevin Chen-Chuan Chang, Yu Lei, and Bo Yang. 2020. Geom-gcn: Geometric graph convolutional networks. *arXiv preprint arXiv:2002.05287* (2020).
- [37] Yifang Qin, Wei Ju, Yiyang Gu, Ziyue Qiao, Zhiping Xiao, and Ming Zhang. 2025. Polycf: Towards optimal spectral graph filters for collaborative filtering. *ACM Transactions on Information Systems* 43, 4 (2025), 1–28.
- [38] Amin Salehi and Hasan Davulcu. 2019. Graph attention auto-encoders. *arXiv preprint arXiv:1905.10715* (2019).
- [39] Yifei Shen, Yongji Wu, Yao Zhang, Caihua Shan, Jun Zhang, B Khaled Letaief, and Dongsheng Li. 2021. How powerful is graph convolution for recommendation?. In *CIKM*. 1619–1629.
- [40] Shashank Sheshar Singh, Samya Muhuri, Shivansh Mishra, Divya Srivastava, Harish Kumar Shakya, and Neeraj Kumar. 2024. Social network analysis: A survey on process, tools, and application. *ACM computing surveys* 56, 8 (2024), 1–39.
- [41] Teague Sterling and John J Irwin. 2015. ZINC 15—ligand discovery for everyone. *Journal of chemical information and modeling* 55, 11 (2015), 2324–2337.
- [42] Fan-Yun Sun, Jordan Hoffmann, Vikas Verma, and Jian Tang. 2019. Infograph: Un-supervised and semi-supervised graph-level representation learning via mutual information maximization. *arXiv preprint arXiv:1908.01000* (2019).
- [43] Qiaoyu Tan, Ninghao Liu, Xiao Huang, Soo-Hyun Choi, Li Li, Rui Chen, and Xia Hu. 2023. S2gae: Self-supervised graph autoencoders are generalizable learners with graph masking. In *WSDM*. 787–795.
- [44] Shantanu Thakoor, Corentin Tallec, Mohammad Gheshlaghi Azar, Rémi Munos, Petar Veličković, and Michal Valko. 2021. Bootstrapped representation learning on graphs. In *ICLR*.
- [45] Yijun Tian, Kaiwen Dong, Chunhui Zhang, Chuxu Zhang, and Nitesh V Chawla. 2023. Heterogeneous graph masked autoencoders. In *AAAI*, Vol. 37. 9997–10005.
- [46] Amanda L Traud, Peter J Mucha, and Mason A Porter. 2012. Social structure of facebook networks. *Physica A: Statistical Mechanics and its Applications* 391, 16 (2012), 4165–4180.
- [47] Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Lio, and Yoshua Bengio. 2017. Graph attention networks. *arXiv preprint arXiv:1710.10903* (2017).
- [48] Petar Veličković, William Fedus, William L Hamilton, Pietro Liò, Yoshua Bengio, and R Devon Hjelm. 2018. Deep graph infomax. *arXiv preprint arXiv:1809.10341* (2018).
- [49] Haorui Wang, Haoteng Yin, Muhan Zhang, and Pan Li. [n. d.]. Equivariant and Stable Positional Encoding for More Powerful Graph Neural Networks. In *ICLR*.
- [50] Liang Wang, Xiang Tao, Qiang Liu, and Shu Wu. 2024. Rethinking graph masked autoencoders through alignment and uniformity. In *AAAI*, Vol. 38. 15528–15536.
- [51] Wenguan Wang, Yi Yang, and Fei Wu. 2024. Towards data-and knowledge-driven AI: a survey on neuro-symbolic computing. *IEEE Transactions on Pattern Analysis and Machine Intelligence* (2024).
- [52] Xiyuan Wang and Muhan Zhang. 2022. How powerful are spectral graph neural networks. In *ICML*. PMLR, 23341–23362.
- [53] Zhibiao Wang, Xiao Wang, Haoyue Deng, Nian Liu, Shirui Pan, and Chunming Hu. 2024. Uncovering the redundancy in graph self-supervised learning models. In *NeurIPS*. 98625–98644.
- [54] Jiancan Wu, Xiang Wang, Fuli Feng, Xiangnan He, Liang Chen, Jianxun Lian, and Xing Xie. 2021. Self-supervised graph learning for recommendation. In *SIGIR*. 726–735.

- [55] Zhenqin Wu, Bharath Ramsundar, Evan N Feinberg, Joseph Gomes, Caleb Geniesse, Aneesh S Pappu, Karl Leswing, and Vijay Pande. 2018. MoleculeNet: a benchmark for molecular machine learning. *Chemical science* 9, 2 (2018), 513–530.
- [56] Jun Xia, Chengshuai Zhao, Bozhen Hu, Zhangyang Gao, Cheng Tan, Yue Liu, Siyuan Li, and Stan Z Li. 2023. Mole-bert: Rethinking pre-training graph neural networks for molecules. (2023).
- [57] Rongwei Xu, Guanfeng Liu, Yan Wang, Xuyun Zhang, Kai Zheng, and Xiaofang Zhou. 2024. Adaptive hypergraph network for trust prediction. In *ICDE. IEEE*, 2986–2999.
- [58] Zhirui Yang, Yulan Hu, Sheng Ouyang, Jingyu Liu, Shuqiang Wang, Xibo Ma, Wenhao Wang, Hanjing Su, and Yong Liu. 2024. WaveNet: tackling non-stationary graph signals via graph spectral wavelets. In *AAAI*, Vol. 38. 9287–9295.
- [59] Yihang Yin, Qingzhong Wang, Siyu Huang, Haoyi Xiong, and Xiang Zhang. 2022. Autogcl: Automated graph contrastive learning via learnable view generators. In *AAAI*, Vol. 36. 8892–8900.
- [60] Yuning You, Tianlong Chen, Yang Shen, and Zhangyang Wang. 2021. Graph contrastive learning automated. In *ICML*. PMLR, 12121–12132.
- [61] Yuning You, Tianlong Chen, Yongduo Sui, Ting Chen, Zhangyang Wang, and Yang Shen. 2020. Graph contrastive learning with augmentations. *Advances in neural information processing systems* 33 (2020), 5812–5823.
- [62] Hengrui Zhang, Qitian Yu, Junchi Yan, David Wipf, and Philip S Yu. 2021. From canonical correlation analysis to self-supervised graph neural networks. *Advances in Neural Information Processing Systems* 34 (2021), 76–89.
- [63] Yige Zhao, Jianxiang Yu, Yao Cheng, Chengcheng Yu, Yiding Liu, Xiang Li, and Shuaiqiang Wang. 2025. Variational Graph Autoencoder for Heterogeneous Information Networks with Missing and Inaccurate Attributes. In *KDD*. 2067–2078.
- [64] Guanghui Zhu, Wang Lu, Chunfeng Yuan, and Yihua Huang. 2023. Adamcl: Adaptive fusion multi-view contrastive learning for collaborative filtering. In *SIGIR*. 1076–1085.
- [65] Ziyun Zou, Yinghui Jiang, Lian Shen, Juan Liu, and Xiangrong Liu. 2025. Loha: Direct graph spectral contrastive learning between low-pass and high-pass views. In *AAAI*, Vol. 39. 13492–13500.

A Model Details

A.1 Training Procedure of FC-GSSL

In this section, we will illustrate the training process of the FC-GSSL in detail. The pseudo-code of FC-GSSL is shown in Algorithm 1.

- **Step 1.** We preprocess the graph, compute the top- K eigenvectors $\mathbf{U}$ and eigenvalues Λ , and then utilize them to calculate the low-frequency signal contributions C_N and C_E , as well as the edge features $\mathbf{E}$, followed by the initialization of the network model (lines 1-2).
- **Step 2.** Within each training epoch, we generate multiple corrupted graphs using different sampling strategies, which are then fed into the encoder to generate and align their node representations (lines 4-8).
- **Step 3.** The generated node representations are fed into the decoders to formulate reconstruction losses (lines 9-12), with the overall loss function being constructed and minimized to update the model parameters (lines 13-14).
- **Step 4.** Return the trained encoder (line 15).

A.2 Time Complexity Analysis

We analyze the time complexity of various components in FC-GSSL from multiple aspects: (1) To improve computational efficiency during eigen-decomposition, we consider only the smallest K eigenvalues, reducing the time complexity to $O(N^2K)$. (2) The time complexity in the frequency contribution analysis stage is $O(|\mathcal{E}|K^2)$, where $|\mathcal{E}|$ represents the number of edges. (3) The time complexity of the encoder is $O(|\mathcal{E}|d)$. (4) Since the decoder is implemented solely using MLP, its time complexity is approximately $O(d^2)$.

Algorithm 1: FC-GSSL

Input: Graph $\mathcal{G} = \{\mathcal{V}, \mathcal{E}, \mathbf{X}\}$, sampling rates r_N and r_E , and epochs ψ .

- 1 **Preprocess:** Compute the top- K eigenvectors $\mathbf{U}$ and eigenvalues Λ , and then utilize them to calculate the low-frequency signal contributions C_N and C_E , as well as the edge features $\mathbf{E}$.
- 2 **Init:** Encoder f_{enc} , decoder f_{dec}^N , decoder f_{dec}^E , and learnable token $\mathbf{x}_{[M]}$.
- 3 **for** $i \leftarrow 1$ **to** ψ **do**
- 4 $\mathcal{P}_N, \mathcal{Q}_N, \mathcal{P}_E, \mathcal{Q}_E \leftarrow$ Generate sampling sets based on contribution values C_N and C_E , with sampling rates r_N and r_E ;
- 5 $\mathcal{S}_N, \mathcal{S}_E, \mathcal{S}_C \leftarrow$ Generate the corrupted item sets from sampling sets $\mathcal{P}_N, \mathcal{Q}_N, \mathcal{P}_E$, and $\mathcal{Q}_E$;
- 6 $\mathcal{G}_N, \mathcal{G}_E, \mathcal{G}_C \leftarrow$ Generate the corrupted corruption graph from the corrupted item sets $\mathcal{S}_N, \mathcal{S}_E$, and $\mathcal{S}_C$;
- 7 $\mathbf{X}_N^{(L)}, \mathbf{X}_E^{(L)}, \mathbf{X}_C^{(L)} \leftarrow$ Using $\mathcal{G}_N, \mathcal{G}_E$, and $\mathcal{G}_C$ as input to the encoder f_{enc} to generate the corresponding node representations;
- 8 $\mathcal{L}_{align} \leftarrow$ Generate an alignment loss by aligning node representations $\mathbf{X}_N^{(L)}, \mathbf{X}_E^{(L)}$, and $\mathbf{X}_C^{(L)}$;
- 9 $\hat{\mathbf{X}} \leftarrow$ Use $\mathbf{X}_N^{(L)}$ as the input of the decoder f_{dec}^N to reconstruct node features;
- 10 $\mathcal{L}_{node} \leftarrow$ Generate the loss for node feature reconstruction based on $\hat{\mathbf{X}}$ and $\mathbf{X}$;
- 11 $\hat{\mathbf{E}} \leftarrow$ Use $\mathbf{E}_N^{(L)}$ as the input of the decoder f_{dec}^E to reconstruct edge features;
- 12 $\mathcal{L}_{edge} \leftarrow$ Generate the loss for edge structure reconstruction based on $\hat{\mathbf{E}}$ and $\mathbf{E}$;
- 13 $\mathcal{L} \leftarrow$ Generate the overall loss from $\mathcal{L}_{node}$, $\mathcal{L}_{edge}$, and $\mathcal{L}_{align}$;
- 14 Update f_{enc} , f_{dec}^N , and f_{dec}^E by minimizing $\mathcal{L}$;
- 15 **return** Trained encoder f_{enc} ;

Table 5: Dataset statistics for node classification.

Dataset	V	E	F	C	Train/Valid/Test
BlogCatalog	5,196	343,486	8,189	6	120/1,000/1,000
Chameleon	2,277	62,792	2,325	5	1,092/729/456
Squirrel	5,201	396,846	2,089	5	2,496/1,664/1,041
Actor	7,600	53,411	932	5	3,648/2,432/1,520
arXiv-year	169,343	2,315,598	128	5	84671/42335/42337
Penn94	41,554	2,724,458	4,814	2	19,407/9,703/9,705

Table 6: Dataset statistics for graph prediction and transfer learning.

Dataset	G	Avg. V	Avg. E	C	Task	Metric
molesol	1,128	13.3	13.7	1	Regression	RMSE
mollipo	4,200	27.0	29.5	1	Regression	RMSE
molfreeolv	642	8.7	8.4	1	Regression	RMSE
molbase	1,513	34.1	36.9	1	Binary Class.	ROC-AUC
molbbbp	2,039	24.1	26.0	1	Binary Class.	ROC-AUC
molcintox	1,477	26.2	27.9	2	Binary Class.	ROC-AUC
moltox21	7,831	18.6	19.3	12	Binary Class.	ROC-AUC
ZINC15	2,000,000	26.62	57.72	-	Pre-Training	-
QM9	1,177,631	8.80	18.81	12 Targets	Finetuning	MAE

B Experimental Details

B.1 Statistics of Datasets

We benchmarked FC-GSSL on tasks including node classification, graph prediction, and transfer learning. Experiments were conducted on 6 node classification and 7 graph-level prediction datasets. For transfer learning, we employed ZINC15 and QM9, splitting the latter into training, validation, and test sets with an 80%/10%/10% ratio according to [32]. Public data splits were adopted for the other datasets. Table 5 presents the statistics of the node classification datasets, with $|V|$, $|E|$, $|F|$, and $|C|$ denoting the number of nodes,

Table 7: Hyperparameters of node classification.

Dataset	lr	α	β	r_N	r_E	K	K_e
BlogCatalog	0.001	0.01	0.00001	0.3	0.3	V	50
Chameleon	0.001	0.01	0.0001	0.3	0.3	V	50
Squirrel	0.001	0.01	0.00001	0.3	0.3	V	50
Actor	0.0005	0.01	0.0001	0.3	0.3	V	50
arXiv-year	0.001	1	0.01	0.1	0.1	1000	100
Penn94	0.0025	1	0.1	0.25	0.25	1000	200

Table 8: Hyperparameters of graph prediction.

Dataset	pooling	lr	α	β	r_N	r_E	K	K_e
molesol	sum	0.0005	0.01	0.00001	0.75	0.75	8	8
mollipo	sum	0.0005	0.001	0.0001	0.25	0.25	30	30
molreesolv	sum	0.00002	0.01	0.0001	0.5	0.25	15	15
molbase	mean	0.001	0.1	0.0001	0.3	0.3	30	30
molbbbp	mean	0.001	0.01	0.00001	0.4	0.4	30	30
molcintox	mean	0.001	0.1	0.001	0.3	0.3	30	30
moltox21	mean	0.0001	0.2	0.00001	0.25	0.25	8	8

edges, feature dimension, and classes, respectively. Table 6 presents the statistics of the datasets used for graph prediction and transfer learning, where $|G|$ represents the number of graphs.

B.2 Hyperparameters

The key hyperparameters for node classification and graph prediction are provided in Table 7 and Table 8, respectively. Specifically, parameter K controls the number of low-frequency signals considered for computing contributions, whereas K_e controls the number used in the generation of positional encodings and edge features. In transfer learning, we pre-trained FC-GSSL for 100 epochs on 2 million molecules sampled from the ZINC15 dataset. During pre-training, α was set to 0.01, β to 0.0001, r_N and r_E both to 0.35, and K and K_e both to 6. In the fine-tuning phase, the encoder and its additional MLP were jointly trained on the QM9 dataset, with an initial learning rate of 0.001. For all experiments, the hyperparameter γ is searched in the range of $\{1.0, 2.0, 3.0\}$, and the hyperparameter τ is set to 0.2. More detailed hyperparameter configurations are available in the source code at <https://github.com/rookitkitlee/FC-GSSL>.